

Structure M1 - multi-operation system design + internal data structure + analytical retrieval + strict output formatting

Structured problem statements based on the provided structure, strict rule validation style, and competitive-coding format.

1. Global Employee Access Log Validator

A corporate security system records employee building access logs that must strictly follow a predefined structure.

Each access log must follow the exact format:

EMP-<EMP_ID>|<ACCESS_DATE>|<ACCESS_TIME>|<LOCATION>|<ACCESS_STATUS>

The system must validate each log using the rules below:

1. Employee ID

- Format must be: EMP-XXXX
- Must contain exactly 4 digits
- First digit must not be 0
- No digit should repeat more than 2 times consecutively

2. Access Date

- Format: YYYY-MM-DD
- Year must be between 2010 and 2030
- Month must be valid (1–12)
- Day must be valid based on month
- February must follow leap year rule (divisible by 4)

3. Access Time

- Format: HH:MM (24-hour format)
- $00 \leq HH \leq 23$
- $00 \leq MM \leq 59$

4. Location

Must be exactly one of: HQ, BRANCH1, BRANCH2, WAREHOUSE, DATA_CENTER

5. Access Status

Must be exactly one of: GRANTED, DENIED, PENDING

Print "VALID ACCESS LOG" if valid, otherwise "INVALID ACCESS LOG".

Function:

validateAccess(String record)

2. Online Banking Transaction Record Validator

A digital banking system processes financial transaction records that must strictly follow compliance regulations.

Each transaction must follow:

TXN-<TXN_ID>|<TXN_DATE>|<ACCOUNT_TYPE>|<AMOUNT>|<STATUS>

1. Transaction ID

- Format: TXN-XXXXXXXX
- Exactly 8 digits
- First digit must not be 0
- No digit should repeat more than 3 times consecutively

2. Transaction Date

- Format: YYYY-MM-DD
- Year must be between 2000–2099
- Proper month/day validation
- Leap year validation required

3. Account Type

Must be one of: SAVINGS, CURRENT, CREDIT, LOAN

4. Amount

- Positive number
- Up to 2 decimal places
- No leading zeros unless exactly 0
- Maximum: 10000000.00

5. Status

Must be one of: SUCCESS, FAILED, PROCESSING, REVERSED

Print "VALID TRANSACTION" or "INVALID TRANSACTION".

Function:

validateTransaction(String record)

3. University Examination Result Validator

A university result processing system validates student exam records before publishing results.

Format:

STU-<ROLL_NO>|<EXAM_DATE>|<SUBJECT_CODE>|<MARKS>|<RESULT_STATUS>

1. Roll Number

- Format: STU-YYYYNNNN
- YYYY = admission year (2015–2025)
- NNNN = 4 digits
- First digit of NNNN must not be 0

2. Exam Date

- Format: YYYY-MM-DD
- Year must match admission year or later
- Proper calendar validation
- Leap year validation required

3. Subject Code

Must match: SUB-XXX

- Exactly 3 uppercase alphabets

4. Marks

- Integer only
- Range: 0–100
- No leading zeros unless exactly 0

5. Result Status

Must be exactly one of: PASS, FAIL, ABSENT, WITHHELD

Print "VALID RESULT" or "INVALID RESULT".

Function:

validateResult(String record)

4. Airline Ticket Booking Record Validator

An airline reservation system validates booking records before confirmation.

Format:

TKT-<BOOKING_ID>|<TRAVEL_DATE>|<CLASS>|<FARE>|<BOOKING_STATUS>

1. Booking ID

- Format: TKT-XXXXXXX
- Exactly 7 digits

- First digit must not be 0
- No digit repeated more than 2 times consecutively

2. Travel Date

- Format: YYYY-MM-DD
- Must be between 2023–2035
- Proper month/day validation
- Leap year rule applies

3. Class

Must be one of: ECONOMY, PREMIUM, BUSINESS, FIRST

4. Fare

- Positive decimal number
- Up to 2 decimal places
- Maximum: 500000.00
- No leading zeros unless exactly 0

5. Booking Status

Must be one of: CONFIRMED, CANCELLED, WAITLISTED

Print "VALID BOOKING" or "INVALID BOOKING".

Function:

validateBooking(String record)

5. IoT Device Telemetry Record Validator

A smart city IoT monitoring system processes telemetry data from connected devices.

Format:

DEV-<DEVICE_ID>|<TIMESTAMP>|<SENSOR_TYPE>|<VALUE>|<DEVICE_STATUS>

1. Device ID

- Format: DEV-XXXXX
- Exactly 5 digits
- First digit must not be 0
- No digit repeated more than 3 times consecutively

2. Timestamp

- Format: YYYY-MM-DDTHH:MM

- Valid date
- Valid 24-hour time
- Year must be between 2020–2035

3. Sensor Type

Must be exactly one of: TEMP, HUMIDITY, PRESSURE, WIND, RAIN

4. Value

- Positive decimal number
- Up to 3 decimal places
- Maximum allowed: 9999.999

5. Device Status

Must be one of: ACTIVE, INACTIVE, ERROR, MAINTENANCE

Print "VALID TELEMETRY" or "INVALID TELEMETRY".

Function:

validateTelemetry(String record)

6. Library Book Inventory Management System

You are building a digital inventory system for a smart library. Each book can have multiple copies, and each copy has a unique CopyID and page count.

The system must manage books and support analytical retrieval operations. Supported Operations -

1. ADD <BookName> <CopyID> <Pages> => Add a new copy of a book.

Rules:

- If the BookName does not exist, create it.
- If the same CopyID already exists for that BookName, ignore the operation.
- Multiple copies per book are allowed.
- Maintain insertion order internally.

2. LIST <BookName> => Retrieve all copies of the book.

Output Rules:

- Sort copies in increasing order of Pages.
- If two copies have the same Pages, sort lexicographically by CopyID.
- If book does not exist, print: Book Not Found
- Output format:
BookName CopyID Pages

3. RECENT <BookName> => Retrieve the most recently added copy.

Rules:

- Based strictly on insertion order.
- Do NOT sort.
- If book does not exist, print: Book Not Found
- Output format:
BookName CopyID Pages

4. TOTAL_PAGES <BookName> => Print total pages across all copies.

Rules:

- Sum all Pages values.
 - If book does not exist, print: Book Not Found
 - Output format:
BookName TotalPages
-

7. Online Course Enrollment System

You are building a course enrollment platform. Each course can have multiple student enrollments, and each enrollment has a StudentID and score.

Supported Operations

1. ENROLL <CourseName> <StudentID> <Score>

Rules:

- If course does not exist, create it.
- If StudentID is already enrolled in that course, ignore.
- Maintain insertion order.

2. SCORES <CourseName>

Output Rules:

- Sort by Score (descending).
- If tie → lexicographically by StudentID.
- If course does not exist → Course Not Found
- Output format:
CourseName StudentID Score

3. LAST_ENROLLED <CourseName>

- Print most recently enrolled student.

- Based on insertion order.
- If not exist → Course Not Found

4. AVERAGE_SCORE <CourseName>

- Print average score (integer division).
- If not exist → Course Not Found
- Output format:
CourseName AverageScore

8. E-Commerce Product Review System

Story: You are designing a product review management system. Each product can have multiple reviews, and each review contains a ReviewID and Rating (1–5).

Supported Operations

1. ADD_REVIEW <ProductName> <ReviewID> <Rating>

Rules:

- If product does not exist → create.
- If same ReviewID exists → ignore.
- Maintain insertion order.

2. REVIEWS <ProductName>

Output Rules:

- Sort by Rating (descending).
- If tie → lexicographically by ReviewID.
- If product not exist → Product Not Found
- Output format:
ProductName ReviewID Rating

3. MOST_RECENT <ProductName>

- Print last added review.
- If not exist → Product Not Found

4. AVG_RATING <ProductName>

- Print average rating (2 decimal precision).
- If not exist → Product Not Found
- Output format:
ProductName AverageRating

9. Company Project Task Tracker

You are building a task tracking system for multiple company projects. Each project can have multiple tasks, and each task has TaskID and EstimatedHours.

Supported Operations

1. ADD_TASK <ProjectName> <TaskID> <EstimatedHours>

Rules:

- If the project does not exist → create.
- If TaskID already exists in that project → ignore.
- Maintain insertion order.

2. TASKS <ProjectName>

Output Rules:

- Sort by EstimatedHours (ascending).
- If tie → lexicographically by TaskID.
- If project not exist → Project Not Found
- Output format:
ProjectName TaskID EstimatedHours

3. LAST_TASK <ProjectName>

- Print most recently added task.
- If not exist → Project Not Found

4. TOTAL_HOURS <ProjectName>

- Print total estimated hours.
- If not exist → Project Not Found
- Output format:
ProjectName TotalHours

10. Music Playlist Version Manager

You are building a playlist management system for a music streaming app. Each playlist can contain multiple song versions, and each version has a SongID and Duration (seconds).

Supported Operations

1. ADD_SONG <PlaylistName> <SongID> <Duration>

Rules:

- If the playlist does not exist → create.
- If SongID already exists in the playlist → ignore.
- Maintain insertion order.

2. PLAYLIST <PlaylistName>

Output Rules:

- Sort by Duration (ascending).
- If tie → lexicographically by SongID.
- If playlist not exist → Playlist Not Found
- Output format:
PlaylistName SongID Duration

3. RECENT_SONG <PlaylistName>

- Print the most recently added song.
- If not exist → Playlist Not Found

4. TOTAL_DURATION <PlaylistName>

- Print total duration of playlist.
- If not exist → Playlist Not Found
- Output format:
PlaylistName TotalDuration

11. Multi-Currency Digital Wallet System

Implement a digital wallet system that supports multiple currencies and inter-wallet transfers with exchange rates.

Each wallet contains:

- walletId (String)
- holderName (String)
- Map<String, Double> balances (currency → balance)
- List<Transaction>

Each transaction contains:

- type (DEPOSIT / WITHDRAW / TRANSFER_IN / TRANSFER_OUT / FREEZE)
- currency
- amount
- resultingBalance

Supported Commands

1. CREATE <walletId> <holderName>

Create a wallet if not already present.

2. DEPOSIT <walletId> <currency> <amount>

- Add an amount to that currency.
- If currency is not present → initialize.
- Print: DEPOSITED <walletId> <currency> <balance>
- If wallet not found → Wallet not found

3. WITHDRAW <walletId> <currency> <amount>

- If sufficient balance → deduct.
- Else → Insufficient balance
- If wallet frozen → Wallet frozen

4. TRANSFER <fromWallet> <toWallet> <currency> <amount>

- Both wallets must exist.
- Sufficient balance required.
- Record TRANSFER_OUT and TRANSFER_IN.
- Print: TRANSFER SUCCESS
- Otherwise appropriate error.

5. FREEZE <walletId>

Freeze wallet → no withdrawals or transfers allowed.

6. HOLDER_SUMMARY <holderName>

- Print all wallets sorted by walletId.
- Print total balance across all currencies.

12. Smart Investment Portfolio Management System

Each investor can own multiple portfolios. Each portfolio holds multiple stocks.

Portfolio:

- portfolioId
- investorName
- Map<String, Integer> stocks (stockSymbol → quantity)
- double cashBalance

Commands -

1. CREATE_PORTFOLIO <portfolioId> <investorName> <initialCash>

2. BUY <portfolioId> <stockSymbol> <quantity> <pricePerUnit>

- Total cost = quantity × pricePerUnit
- Deduct from cashBalance
- Add stock
- If insufficient cash → Insufficient funds

3. SELL <portfolioId> <stockSymbol> <quantity> <pricePerUnit>

- Must own sufficient quantity
- Increase cashBalance
- If not owned → Stock not found

4. PORTFOLIO_VALUE <portfolioId>

- Print total value (cash + stockValue)
- Assume the latest trade price is the current price.

5. TOP_INVESTOR

- Print investor with highest total portfolio value.

13. Airline Reservation & Waitlist Management System

Each flight contains:

- flightNumber
- totalSeats
- bookedPassengers (List)
- waitlistQueue (Queue)

Passenger:

- passengerId
- seatNumber
- status (CONFIRMED / WAITLISTED)

Commands

1. CREATE_FLIGHT <flightNumber> <seatCapacity>

2. BOOK <flightNumber> <passengerId>

- If seats available → CONFIRMED

- Else → WAITLISTED

3. CANCEL <flightNumber> <passengerId>

- Remove passenger.
- If confirmed seat freed → move first waitlisted passenger to confirmed.

4. FLIGHT_STATUS <flightNumber>

- Print confirmed passengers sorted by seatNumber.
- Then print the waitlist in order.

5. PASSENGER_STATUS <passengerId>

- Search across all flights.

14. Distributed Task Scheduling System

Each worker node:

- nodeId
- capacity (max tasks)
- currentLoad
- List<Task>

Task:

- taskId
- priority (1–5)
- status (QUEUED / RUNNING / COMPLETED)

Commands

1. REGISTER_NODE <nodeId> <capacity>

2. SUBMIT_TASK <taskId> <priority>

- Assign to the node with lowest load.
- If tie → lexicographically smaller nodeId.
- If all full → All nodes busy

3. COMPLETE_TASK <nodeId> <taskId>

- Mark completed
- Reduce load

4. NODE_STATUS <nodeId>

- Print tasks sorted by priority (descending).

5. SYSTEM_LOAD

- Print total tasks running in system.

15. Multi-Level Subscription Streaming Platform

Each user:

- userId
- subscriptionType (FREE / SILVER / GOLD / PLATINUM)
- watchHistory (List<Content>)
- monthlyWatchTime

Content:

- contentId
- duration
- minimumSubscriptionRequired

Commands

1. REGISTER_USER <userId> <subscriptionType>
2. WATCH <userId> <contentId> <durationWatched>

Rules:

- User must meet a minimum subscription level.
- Update monthlyWatchTime.
- FREE users limited to 300 mins/month.
- Print: WATCH SUCCESS
- Else appropriate error.

3. UPGRADE <userId> <newSubscriptionType>

- Upgrade only allowed (no downgrade).

4. TOP_VIEWER

- Print user with highest monthlyWatchTime.

5. RESET_MONTH

- Reset watch time for all users.

16. Enterprise Log Sanitization Engine

A cybersecurity platform must sanitize enterprise server logs before archival.

The system processes N log lines.

You must detect and transform sensitive patterns using **regex with capturing groups and backreferences**.

Redaction Rules (Must Use Regex)

Rule 1 — Mask IPv4 Addresses

Pattern:

IP:<0-255>.<0-255>.<0-255>.<0-255>

Example:

IP:192.168.1.45

Transformation:

IP:..***.45

- Mask the first three octets.
- Preserve the last octet using a capturing group.
- Must validate numeric range via regex (0–255).

Rule 2 — Mask Email Username

Pattern:

Email:<username>@<domain>

Example:

Email:john.doe@company.com

Transformation:

Email:****@company.com

- Mask the entire username.
- Preserve domain using capturing group.

Rule 3 — Collapse Repeated Error Codes

If error code repeats consecutively:

Example:

ERROR404 ERROR404 ERROR404

Replace with:

ERROR404

Must use a backreference group.

Rule 4 — Normalize Timestamp Format

Pattern:

[DD/MM/YYYY HH:MM:SS]

Convert to:

[YYYY-MM-DD HH:MM:SS]

Must use capturing groups and rearrangement.

Processing Order:

1. Mask IP
 2. Mask Email
 3. Collapse error codes
 4. Normalize timestamp
-

17. Financial Transaction Scrubbing Engine

A financial audit system must scrub sensitive transaction logs.

Redaction Rules

Rule 1 — Mask Credit Card Numbers

Pattern:

`\b\d{4}-\d{4}-\d{4}-\d{4}\b`

Transformation:

`****_****_****-<last4>`

Must use capturing groups to retain the last 4 digits.

Rule 2 — Mask IFSC Codes

Pattern:

`IFSC:[A-Z]{4}0\d{6}`

Transformation:

`IFSC:XXXX0*****`

Preserve the mandatory 0 in 5th position.

Rule 3 — Detect & Remove Duplicate Transaction IDs

Pattern:

`TXN\d{6} repeated consecutively`

Example:

`TXN123456 TXN123456`

Replace with single occurrence.

Rule 4 — Standardize Currency Format

Pattern:

`USD <amount>`

Convert to:

`$<amount>`

Must preserve decimal digits.

Processing Order:

1. Mask Credit Cards
 2. Mask IFSC
 3. Remove duplicate TXNs
 4. Normalize currency
-

18. Healthcare Record De-Identification Engine

A hospital compliance engine must redact patient identifiers.

Redaction Rules: Rule 1 — Mask Patient MRN

Pattern:

MRN-[A-Z]{2}\d{6}

Transformation:

MRN-XX*****

Rule 2 — Mask Phone Numbers

Pattern:

(\+91[-])?[6-9]\d{9}

Transformation:

Must handle optional +91 prefix.

Rule 3 — Normalize Repeated Medical Terms

If term repeats consecutively (case insensitive):

Example:

"critical CRITICAL stable"

Replace with:

"critical stable"

Rule 4 — Trim Excessive Whitespace

Replace 3+ spaces with exactly one space.

Must use a regex quantifier.

Processing Order:

1. MRN mask
2. Phone mask
3. Remove duplicate medical terms
4. Normalize whitespace

19. Source Code Cleanup Engine

A static analysis tool must clean sensitive source code before sharing externally.

Redaction Rules

Rule 1 — Mask API Keys

Pattern:

```
API_KEY\s*=\s*" [A-Za-z0-9]{20,}"
```

Transformation:

```
API_KEY="*****"
```

Preserve variable names using capturing groups.

Rule 2 — Mask Hardcoded Passwords

Pattern:

```
password\s*=\s*".*?"
```

Replace value only.

Rule 3 — Remove Duplicate Imports

If same import appears consecutively:

Example:

```
import java.util.*;
```

```
import java.util.*;
```

Replace with a single occurrence using backreference.

Rule 4 — Remove Trailing Semicolon Flood

If line ends with 2+ semicolons (;; or more)

Replace with single ;

Processing Order Strict.

20. Social Media Moderation Filter Engine

A content moderation engine must sanitize user posts.

Redaction Rules

Rule 1 — Mask Mentioned Phone Numbers

Pattern:

```
\b\d{10}\b
```

Transformation:

```
[PHONE REDACTED]
```

Rule 2 — Mask URLs but Preserve Domain

Pattern:

`https?://(www\.)?([^\s]+)(/[^\s]*)?`

Transformation:

`[LINK:\2]`

Must capture and preserve domain only.

Rule 3 — Reduce Character Flooding

If a character repeats 4+ times:

Example:

"Noooooooo wayyyyy!!!!"

Transform to:

"Nooo wayy!!!"

Limit repetition to max 3 occurrences.

Rule 4 — Normalize Repeated Hashtags

Example:

#Sale #Sale #Sale

Replace with single #Sale.

Must use backreference.

Processing Order:

1. Mask phone
 2. Mask URL
 3. Reduce character flooding
 4. Normalize hashtags
-